AUTOTEST STUDIO ENTERPRISE

Tài liệu Kỹ thuật: Hướng dẫn Triển khai & Vận hành Môi trường Mạng LAN Nội bộ (Offline 100%)

Phiên bản tài liệu: 2.0 (Bản cập nhật Kiến trúc Đồng bộ LLM)
Môi trường mục tiêu: Linux Server (Ubuntu, Debian, RHEL, CentOS)
Cổng dịch vụ mặc định: `39793` (Đã ánh xạ từ container 3000)
Khả năng mạng: Không cần kết nối Internet (Isolated LAN)
Docker Compose: Tương thích v2.2 (hỗ trợ legacy 1.17.1 & Compose v2)
Mô hình AI: LiteLLM / vLLM / Ollama (Qwen, LLaMA, DeepSeek...)

1. TỔNG QUAN KIẾN TRÚC HỆ THỐNG

AutoTest Studio là nền tảng tự động hóa kiểm thử web thế hệ mới, tích hợp trí tuệ nhân tạo (AI Engine) và trình duyệt Chromium Headless để diễn giải kịch bản kiểm thử bằng ngôn ngữ tự nhiên thành các chuỗi hành động click, nhập liệu và kiểm tra trên giao diện web.

> **Kiến trúc hợp nhất AI Client nội bộ (Single Unified LLM Bridge)**
> Trong phiên bản này, toàn bộ quá trình xử lý AI được hợp nhất qua một cổng kết nối duy nhất (`LLM_BASE_URL`). Thư viện Stagehand không còn tự động gọi ra cloud mà chuyển tiếp toàn bộ request qua hàm `ClientLLM` nội bộ, chấm dứt hoàn toàn nguy cơ đứt gãy mạng `failed to fetch` khi chạy offline.

Hệ thống bao gồm 3 phân lớp chính gói gọn trong 1 Docker Image duy nhất:

- **Giao diện Web (Frontend):** Ứng dụng React + Vite được biên dịch tĩnh, nhúng trực tiếp vào Backend Express phục vụ qua cổng `3000` (ánh xạ ra cổng ngoài `39793`).
- **Máy chủ điều khiển (Backend Node.js 22):** Tiếp nhận yêu cầu, phân loại kịch bản (Planner) và điều khiển trình duyệt qua Playwright CDP.
- **Bộ não AI Trình duyệt (Stagehand Browser Automation):** Định vị phần tử DOM theo ngữ nghĩa, thực hiện click, điền dữ liệu (Fill/Type), trích xuất và chụp ảnh toàn bộ tiến trình tương tác (Visual Timeline).

2. THÀNH PHẦN GÓI CÀI ĐẶT (autotest-offline.tar.gz)

Gói cài đặt `autotest-offline.tar.gz` (dung lượng ~348 MB) chứa toàn bộ những gì cần thiết để vận hành hệ thống độc lập:

Tệp tin / Thư mục	Định dạng	Mô tả chi tiết
`autotest-studio-offline.tar`	DOCKER IMAGE	Image Linux (Node.js 22 + Chromium + Font Unicode + Backend & Frontend).
`docker-compose.yml`	CẤU HÌNH	Tương thích chuẩn Compose 2.2 (chạy tốt trên `docker-compose 1.17.1`).
`deploy.sh`	BASH SCRIPT	Script tự động hóa 5 bước: tạo thư mục, nạp image tar, kiểm tra cổng 39793, và khởi chạy.
`.env` & `.env.example`	BIẾN MÔI TRƯỜNG	Chứa cổng 39793, địa chỉ IP máy chủ LLM nội bộ, tên model AI và khóa API.
`data/` & `screenshots/`	Thư mục dữ liệu	Lưu trữ dữ liệu lịch sử chạy test và ảnh chụp màn hình từng bước tương tác.
`README.md`	Tài liệu	Bản tóm tắt hướng dẫn thao tác nhanh trên terminal Linux.

3. QUY TRÌNH TRIỂN KHAI TỪNG BƯỚC TRÊN LINUX SERVER

Bước 1: Sao chép gói cài đặt sang máy chủ

Dùng lệnh `scp` từ máy tính cá nhân để đẩy tệp nén sang máy chủ Linux thông qua giao thức SSH:

```
# Cú pháp sao chép file sang server:
scp autotest-offline.tar.gz user@<IP-SERVER>:/home/user/
```

Bước 2: Đăng nhập SSH và giải nén

```
ssh user@<IP-SERVER>
# Giải nén gói triển khai
tar -xzf autotest-offline.tar.gz
cd offline-package
```

Bước 3: Cấu hình thông số môi trường (.env)

Mở tệp `.env` để tùy chỉnh kết nối tới model AI trong mạng LAN của bạn:

```
nano .env
```

Cấu hình mẫu chuẩn xác cho mô hình mạng nội bộ dùng model Qwen / LiteLLM / vLLM:

```
# ========================================================
# AutoTest Studio - Cấu hình mô hình AI & Triển khai
# ========================================================

# Khóa API của LLM (nếu server local yêu cầu virtual key như LiteLLM thì điền sk-..., nếu không cần thì c
LLM_API_KEY=sk-my-virtual-key

# Địa chỉ endpoint LLM nội bộ hỗ trợ chuẩn OpenAI-compatible (/chat/completions)
LLM_BASE_URL=http://192.168.205.151:5000/api

# Tên Model AI dùng chung cho toàn bộ hệ thống (phân rã kịch bản + click/type tự động)
LLM_MODEL=qwen3.8-27b

# Timeout khi gọi LLM (milliseconds)
LLM_TIMEOUT_MS=60000

# Cấu hình Cổng truy cập ngoài (Khách hàng mở trên trình duyệt)
EXTERNAL_PORT=39793
PORT=3000
HOST=0.0.0.0
HEADLESS=true
```

Bước 4: Chạy script triển khai tự động

Chạy script `deploy.sh` với quyền quản trị:

```
sudo bash deploy.sh
```

> **Script deploy.sh sẽ tự động thực hiện hoàn toàn:**
> 1. Kiểm tra engine Docker Daemon trên máy chủ.
> 2. Cấp quyền `chmod -R 777` cho thư mục `data` và `screenshots`.
> 3. Kiểm tra cổng ngoài `39793` có bị ứng dụng khác chiếm dụng không.
> 4. Nạp image `autotest-studio-offline.tar` vào Docker.
> 5. Khởi chạy container bằng `docker-compose` (chuẩn Compose 2.2 tương thích bản 1.17.1) hoặc tự động dùng lệnh `docker run` dự phòng.

Bước 5: Truy cập hệ thống

Mở trình duyệt web từ bất kỳ máy tính nào trong mạng LAN và truy cập:

```
http://<IP-CỦA-SERVER-LINUX>:39793
```

4. CƠ CHẾ KẾT NỐI AI & BỘ LỌC SUY LUẬN <think>

Sự khác biệt giữa Giai đoạn 1 (Planner) và Giai đoạn 2 (Stagehand)

Hệ thống xử lý kịch bản kiểm thử qua 2 giai đoạn độc lập nhưng dùng chung 1 nguồn AI duy nhất:

- **Giai đoạn 1 (Planner - Phân rã kịch bản):** Gửi câu lệnh bằng tiếng Việt/Anh tới `LLM_BASE_URL` để phân tích thành danh sách các bước có cấu trúc (ví dụ: 1. `goto`, 2. `act`, 3. `observe`).
- **Giai đoạn 2 (Executor - Tương tác thực tế trên Web):**
  - Bước `goto`: Dùng lệnh Chromium thuần để mở URL (không gọi AI).
  - Bước `act` (Ví dụ: "*enter username admin*") hoặc `observe`: Stagehand trích xuất cây cấu trúc DOM của trang web, gửi về cho model `LLM_MODEL` qua giao tiếp `ClientLLM` nội bộ để xác định chính xác ô nhập/nút bấm và thực hiện hành động.

Tại sao cần bộ lọc thẻ <think> cho model Qwen & DeepSeek?

Các dòng mô hình mã nguồn mở hiện đại (Qwen 2.5/3.8, DeepSeek-R1) khi nhận dữ liệu thường tiến hành "suy nghĩ nội tâm" trước khi trả kết quả, kẹp trong cặp thẻ `<think> ... </think>`.

Thư viện Stagehand kiểm tra dữ liệu bằng bộ luật Zod Schema nghiêm ngặt. Nếu trong chuỗi phản hồi còn dính các ký tự chữ suy nghĩ hoặc thẻ `<think>`, lệnh đọc JSON sẽ bị lỗi cú pháp dẫn đến `ZodError` hoặc `EXECUTION_ERROR`.

> **Giải pháp đã tích hợp sẵn:**
> Hàm `callUnifiedLLM` trong backend tự động áp dụng biểu thức chính quy (Regex) cắt bỏ toàn bộ khối `<think>...</think>`, đồng thời tự động chuẩn hóa cấu trúc mảng `elements: []` để Stagehand luôn nhận được dữ liệu chuẩn xác 100%.

5. BẢNG CÁC LỆNH QUẢN TRỊ VẬN HÀNH THƯỜNG DÙNG

Tác vụ quản trị	Câu lệnh thực thi trên Linux Server
Xem log hoạt động thời gian thực	`docker logs -f autotest-studio`
Xem 100 dòng log gần nhất	`docker logs --tail 100 autotest-studio`
Kiểm tra trạng thái Container	`docker ps --filter "name=autotest-studio"`
Khởi động lại dịch vụ	`docker restart autotest-studio` (hoặc `docker-compose restart`)
Dừng dịch vụ	`docker-compose down` (hoặc `docker stop autotest-studio`)
Thay đổi cấu hình .env	Chỉnh sửa `nano .env` rồi chạy `docker restart autotest-studio` (Có thể sửa trực tiếp trên giao diện Web mà không cần SSH).

6. XỬ LÝ SỰ CỐ THƯỜNG GẶP (TROUBLESHOOTING)

1. Lỗi: "Model trả mã HTTP 401: LiteLLM Virtual Key expected"

Nguyên nhân: Server LiteLLM của bạn đang bật chế độ bảo mật yêu cầu Virtual Key (bắt đầu bằng `sk-...`) nhưng trong file `.env` bạn lại điền key của Groq (bắt đầu bằng `gsk_...`).

Khắc phục: Mở `.env`, sửa biến `LLM_API_KEY=sk-...` bằng Virtual Key được cấp trên LiteLLM dashboard của bạn.

Trang 5

2. Lỗi: "Cổng 39793 đang bị chiếm dụng"

Khắc phục: Mở file `.env`, đổi dòng `EXTERNAL_PORT=39793` thành cổng khác (ví dụ: `39794`) rồi chạy lại `sudo bash deploy.sh`.

3. Lỗi: "Unsupported Compose file version" trên docker-compose 1.17.1

Khắc phục: File `docker-compose.yml` đi kèm trong gói này đã được khóa ở chuẩn `version: "2.2"`. Nếu bạn tự tạo file compose mới, tuyệt đối không dùng `version: "3.8"` mà phải giữ `version: "2.2"`.

AutoTest Studio Enterprise Edition • Bản quyền tài liệu lưu hành nội bộ • Triển khai mạng LAN Offline